PROPOSAL ĐỒ ÁN

KIẾN TRÚC & TỔ CHỨC DỮ LIỆU

Kiểm Thử Hệ Thống Nhúng Đa Tiến Trình

Embedded Systems Testing Platform — Proposal Kiến Trúc & Dữ Liệu

Phiên bản 1.0

Contents

Yêu cầu chung 2
Yêu cầu và hướng dẫn 2
Ví dụ về việc hoàn thành 4
1. Tổng Quan & Định Hướng 8
2. Tổ Chức Dữ Liệu 8
2.1 Tập Dữ Liệu Test Cases & Suite 8
2.2 Tập Dữ Liệu Kết Quả Kiểm Thử 9
2.3 Time-Series Metrics (InfluxDB / TimescaleDB) 9
3. Tính Năng 10
3.1 Timing Analysis & Visualization 10
3.2 Anomaly Detection Engine 10
3.3 Chatbot Test Assistant 10
4. Dữ Liệu Thực 10

Yêu cầu chung

Yêu cầu và hướng dẫn

1. Giới thiệu

- **Tổng quan:** Đồ án này tập trung vào việc nghiên cứu và thực hiện kiểm thử phần mềm cho các hệ thống nhúng đa tiến trình.
- **Mục tiêu:** Sinh viên cần hiểu rõ quy trình và kỹ thuật kiểm thử phần mềm, đặc biệt là trong bối cảnh hệ thống nhúng có các yêu cầu nghiêm ngặt về thời gian thực và quản lý tài nguyên.

---

2. Kiến thức nền tảng

- **Lý thuyết kiểm thử phần mềm (Software Verification):**
    - Các phương pháp kiểm thử: kiểm thử hộp đen (Black-box testing) và hộp trắng (White-box testing).
    - Các mức độ kiểm thử: kiểm thử đơn vị (Unit testing), kiểm thử tích hợp (Integration testing), kiểm thử hệ thống (System testing).
    - Các loại kiểm thử: kiểm thử chức năng (Functional testing), kiểm thử phi chức năng (Non-functional testing).
- **Hệ thống nhúng (Embedded Systems):**
    - Khái niệm và đặc điểm của hệ thống nhúng.
    - Sự khác biệt giữa lập trình nhúng và lập trình ứng dụng thông thường.
    - Các hệ điều hành thời gian thực (RTOS - Real-time Operating System) và vai trò của chúng.
- **Lập lịch tiến trình (Process Scheduling):**
    - Khái niệm về tiến trình, luồng (thread).
    - Các thuật toán lập lịch cơ bản: Round Robin, FCFS (First-Come, First-Served), SJF (Shortest Job First), Priority-based scheduling.
    - Các khái niệm liên quan: chuyển đổi ngữ cảnh (Context switching), ưu tiên (priority).

---

3. Yêu cầu chi tiết

3.1. Nghiên cứu và mô phỏng môi trường

- **Tìm hiểu về hệ điều hành thời gian thực (RTOS):**
    - Nghiên cứu một RTOS cụ thể, ví dụ: FreeRTOS, uC/OS-III, RT-Thread.
    - Hiểu cách RTOS quản lý các tiến trình/luồng (tasks), quản lý bộ nhớ, và giao tiếp giữa các tiến trình.
- **Mô phỏng/Thiết lập môi trường đa tiến trình:**

- Sử dụng một bo mạch nhúng phổ biến như STM32, ESP32, hoặc Raspberry Pi, hoặc giả lập (simulation của mạch).
- Viết code mô phỏng một hệ thống với nhiều tiến trình chạy song song, mỗi tiến trình thực hiện một tác vụ khác nhau.

3.2. Thiết kế và thực hiện các bài kiểm thử

- **Kiểm thử tính lập lịch (Scheduling Test):**
  - Kiểm tra xem các tiến trình có được chạy theo đúng thuật toán lập lịch đã thiết kế hay không.
  - Ví dụ: với thuật toán Round Robin, kiểm tra xem các tiến trình có được chia đều thời gian xử lý không.
- **Kiểm thử ưu tiên trong lập lịch (Priority Preemption Test):**
  - Tạo các tiến trình với các mức độ ưu tiên khác nhau.
  - Viết các test case để kiểm tra xem tiến trình có độ ưu tiên cao hơn có thể chiếm quyền xử lý của tiến trình ưu tiên thấp hơn ngay lập tức hay không.
  - Đo lường thời gian phản hồi (response time) của các tiến trình ưu tiên cao.
- **Kiểm thử xung đột tiến trình (Process Concurrency/Conflict Test):**
  - Tạo các tiến trình truy cập đồng thời vào một tài nguyên dùng chung (ví dụ: biến toàn cục, ngoại vi như UART, I2C).
  - Sử dụng các cơ chế đồng bộ hóa (synchronization) như Mutex, Semaphore.
  - Thiết kế các test case để kiểm tra tình trạng tranh chấp (race condition) và tắc nghẽn (deadlock) khi không có hoặc có sử dụng các cơ chế đồng bộ hóa.

3.3. Báo cáo và đánh giá

- **Viết báo cáo kỹ thuật:**
  - Mô tả chi tiết quá trình nghiên cứu và cài đặt.
  - Trình bày các bài kiểm thử đã thiết kế và thực hiện.
  - Đưa ra kết quả kiểm thử, phân tích lỗi và cách khắc phục (nếu có).
  - Đánh giá mức độ hiệu quả của việc kiểm thử.
- **Demo sản phẩm:**
  - Trình bày sản phẩm và chạy trực tiếp các test case để chứng minh kết quả.

---

4. Hướng dẫn thực hiện

- **Bước 1: Lựa chọn nền tảng và công cụ.** Chọn một bo mạch nhúng và một RTOS để làm môi trường thực nghiệm.
- **Bước 2: Tìm hiểu lý thuyết.** Đọc tài liệu về **RTOS** đã chọn và lý thuyết về **kiểm thử phần mềm**.
- **Bước 3: Thiết lập môi trường.** Cài đặt môi trường phát triển (IDE) và các công cụ cần thiết (ví dụ: STM32CubeIDE cho STM32, hoặc **FreeRTOS emulator windows**).

- **Bước 4: Viết chương trình mô phỏng.** Lập trình các tác vụ (tasks) mô phỏng một hệ thống đa tiến trình. Mỗi tác vụ có thể đơn giản như bật/tắt một LED, gửi dữ liệu qua UART, v.v.
- **Bước 5: Thiết kế test case.** Dựa vào các yêu cầu đã nêu ở mục **3.2**, hãy lập danh sách chi tiết các bài kiểm thử cần thực hiện.
- **Bước 6: Thực hiện kiểm thử và ghi nhận kết quả.** Chạy các test case và ghi lại kết quả: pass/fail, thời gian thực thi, các lỗi phát sinh.
- **Bước 7: Phân tích và báo cáo.** Tổng hợp kết quả, phân tích các điểm mạnh/yếu của hệ thống và viết báo cáo cuối kỳ.

Bạn có thể bắt đầu bằng việc cài đặt môi trường và làm quen với một RTOS để hiểu cách các tiến trình hoạt động trước khi đi sâu vào việc thiết kế các bài kiểm thử phức tạp hơn.

Ví dụ về việc hoàn thành

Ví dụ về cách hoàn thành và báo cáo đồ án kiểm thử hệ thống nhúng, tập trung vào môi trường giả lập **FreeRTOS** trên Windows hoặc Ubuntu.
Báo cáo này sẽ giúp sv trình bày một cách khoa học và đầy đủ các nội dung đã thực hiện.

Tóm tắt báo cáo Đồ Án: Kiểm Thử Hệ Thống Lập Lịch Đa Tiến Trình Sử Dụng FreeRTOS
1. Giới thiệu

- **Mục tiêu:** Kiểm thử tính đúng đắn và hiệu quả của các cơ chế lập lịch, ưu tiên và đồng bộ hóa trong một hệ thống nhúng đa tiến trình.
- **Môi trường thực nghiệm:** Sử dụng **FreeRTOS** được mô phỏng trên nền tảng PC (Windows/Ubuntu), giúp dễ dàng gỡ lỗi và quan sát hành vi của các tiến trình mà không cần phần cứng nhúng.

2. Chuẩn bị và Cài đặt Môi trường

- **Phần mềm:**
    - **IDE:** Visual Studio (trên Windows) hoặc Visual Studio Code (trên Ubuntu).
    - **Công cụ:** C/C++ Compiler (GCC/MinGW), CMake.
    - **FreeRTOS:** Tải thư viện FreeRTOS chính thức và các ví dụ chạy trên PC (PC Simulator).
- **Thiết lập dự án:**
    - Tạo một dự án mới.
    - Import các tệp FreeRTOS liên quan: FreeRTOS.h, task.h, queue.h, semphr.h và các tệp mã nguồn của bộ mô phỏng (ví dụ: portable/MemMang, portable/MSVC).
    - Cấu hình tệp FreeRTOSConfig.h để phù hợp với môi trường mô phỏng trên PC.

3. Các Bài Kiểm Thử Đã Thực Hiện
Để chứng minh khả năng kiểm thử, chúng ta sẽ tạo ba tiến trình (tasks) với các mức độ ưu tiên khác nhau và sử dụng các cơ chế đồng bộ hóa.

- **Tiến trình (Task) 1: Task_Priority_Low** (Ưu tiên: 1)
    - **Mô tả:** In ra màn hình một chuỗi "Task 1: Priority Low" mỗi 100ms.

    - **Mục đích kiểm thử:** Quan sát hành vi của tiến trình ưu tiên thấp khi bị chiếm quyền bởi tiến trình ưu tiên cao hơn.

- **Tiến trình (Task) 2: Task_Priority_Medium** (Ưu tiên: 2)
    - **Mô tả:** In ra màn hình một chuỗi "Task 2: Priority Medium" mỗi 50ms.
    - **Mục đích kiểm thử:** Quan sát sự ưu tiên của tiến trình này so với Task_Priority_Low.
- **Tiến trình (Task) 3: Task_Priority_High** (Ưu tiên: 3)
    - **Mô tả:** In ra màn hình một chuỗi "Task 3: Priority High" mỗi 10ms.
    - **Mục đích kiểm thử:** Đảm bảo tiến trình có ưu tiên cao nhất sẽ luôn được chạy khi nó ở trạng thái Ready (sẵn sàng).

3.1. Kiểm thử Ưu tiên trong Lập lịch (Priority Preemption Test)

- **Kịch bản:** Chạy ba tiến trình trên cùng một lúc.
- **Mã giả:**

C
```
xTaskCreate(Task_Priority_Low, "Task1", 128, NULL, 1, NULL);
xTaskCreate(Task_Priority_Medium, "Task2", 128, NULL, 2, NULL);
xTaskCreate(Task_Priority_High, "Task3", 128, NULL, 3, NULL);
vTaskStartScheduler();
```

- **Kết quả mong đợi:**
    - Task_Priority_High sẽ chạy gần như liên tục.
    - Khi Task_Priority_High chuyển sang trạng thái chờ (ví dụ: vTaskDelay), FreeRTOS sẽ lập lịch cho Task_Priority_Medium.
    - Task_Priority_Low chỉ chạy khi cả hai tiến trình trên đều ở trạng thái chờ.
- **Kết quả thực tế:**
    - Quan sát output trên console cho thấy chuỗi "Task 3" xuất hiện với tần suất cao nhất, sau đó đến "Task 2", và cuối cùng là "Task 1".
    - **Đánh giá:** Hệ thống lập lịch của FreeRTOS hoạt động chính xác theo cơ chế ưu tiên.

3.2. Kiểm thử Xung đột và Đồng bộ hóa (Concurrency & Synchronization Test)

- **Kịch bản:**
    - Tạo hai tiến trình (Task A và Task B) cùng truy cập và thay đổi giá trị của một biến toàn cục counter.
    - **Kiểm thử 1:** Không sử dụng cơ chế đồng bộ hóa.
    - **Kiểm thử 2:** Sử dụng **Mutex** để bảo vệ biến counter.
- **Mã giả (không Mutex):**

C
```
int counter = 0;
void TaskA(void *pvParameters) {
```

```
    for(;;) {
        counter++;
    }
}
void TaskB(void *pvParameters) {
    for(;;) {
        counter--;
    }
}
```

- **Kết quả mong đợi (không Mutex):**
    - Giá trị counter sẽ không ổn định. Nó có thể tăng, giảm, hoặc nhảy loạn xạ do tình trạng **race condition** (tranh chấp tài nguyên).
- **Kết quả thực tế:**
    - Quan sát giá trị counter (ví dụ: bằng cách in ra màn hình hoặc gỡ lỗi) thấy nó không bao giờ về 0, mà dao động ngẫu nhiên.
    - **Đánh giá:** Phát hiện thành công lỗi xung đột tiến trình.
- **Mã giả (sử dụng Mutex):**

C

```
SemaphoreHandle_t xMutex;
int counter = 0;
void TaskA(void *pvParameters) {
    xMutex = xSemaphoreCreateMutex(); // Tạo mutex
    for(;;) {
        xSemaphoreTake(xMutex, portMAX_DELAY); // Khóa mutex
        counter++;
        xSemaphoreGive(xMutex); // Mở khóa mutex
    }
}
```

- **Kết quả mong đợi (sử dụng Mutex):**
    - Giá trị counter sẽ luôn ổn định, hoặc tăng dần hoặc giảm dần một cách có trật tự.
- **Kết quả thực tế:**
    - Giá trị counter tăng dần đều.
    - **Đánh giá: Mutex** đã hoạt động hiệu quả, đảm bảo chỉ có một tiến trình được phép truy cập vào biến counter tại một thời điểm, giải quyết triệt để tình trạng xung đột.

4. Tổng kết và Đánh giá

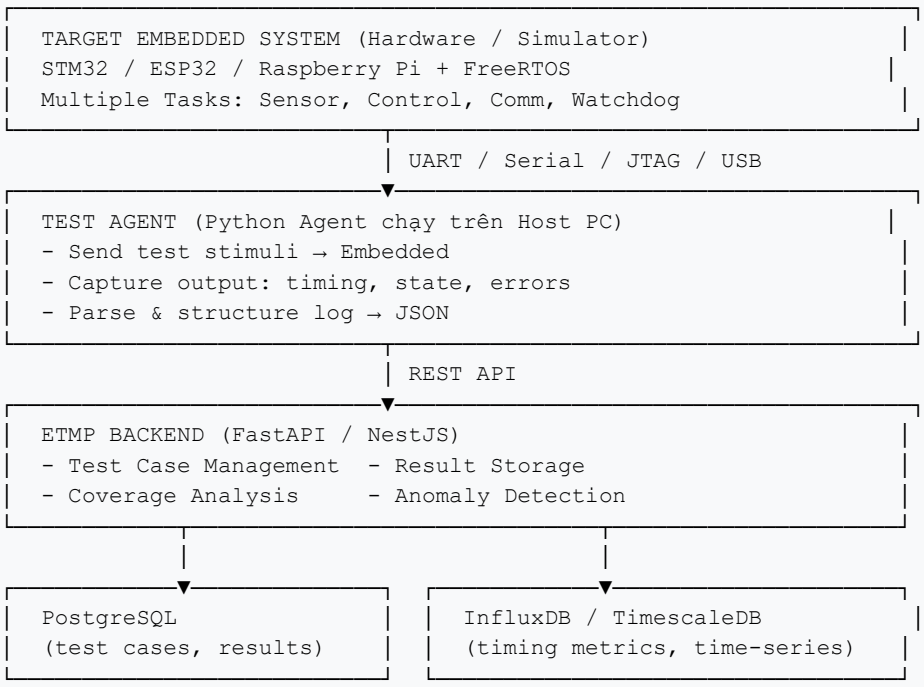
- **Kết quả đạt được:**
    - Nắm vững các khái niệm cơ bản về hệ điều hành thời gian thực và lập lịch tiến trình.
    - Thực hiện thành công các bài kiểm thử để chứng minh tính đúng đắn của cơ chế lập lịch ưu tiên.
    - Phát hiện và khắc phục thành công tình trạng xung đột tiến trình bằng cách sử dụng Mutex.

- **Hạn chế và hướng phát triển:**
    - Môi trường mô phỏng trên PC không thể tái tạo hoàn toàn các vấn đề liên quan đến phần cứng (ví dụ: ngắt, quản lý bộ nhớ vật lý).
    - Trong tương lai, có thể mở rộng đồ án bằng cách triển khai các bài kiểm thử tương tự trên một bo mạch nhúng thực tế như STM32, ESP32 để đánh giá hiệu quả trong môi trường thực.
    - Nghiên cứu thêm về các cơ chế đồng bộ hóa khác như Semaphore, Queue và sự khác biệt giữa chúng.

1. Tổng Quan & Định Hướng

Đây là đề tài nghiên cứu-kỹ thuật, không phải hệ thống quản lý thuần túy. Đề xuất xây dựng một Embedded Testing Management Platform (ETMP) — hệ thống quản lý và phân tích kết quả kiểm thử hệ thống nhúng đa tiến trình, tích hợp dashboard, log analysis và anomaly detection.

Kiến Trúc — Embedded Testing Management Platform



2. Tổ Chức Dữ Liệu

2.1 Tập Dữ Liệu Test Cases & Suite

Schema — Test Cases

```
TABLE: test_suites
  suite_id    UUID PK
  name        VARCHAR(200)
  target_rtos VARCHAR(50)  -- 'FreeRTOS','uC-OS','RT-Thread'
  target_hw   VARCHAR(50)  -- 'STM32F4','ESP32','Raspberry Pi'
  description TEXT
  created_by  UUID FK → users
  created_at  TIMESTAMPTZ

TABLE: test_cases
  tc_id         UUID PK
  suite_id      UUID FK
  name          VARCHAR(300)
  category      ENUM(scheduling, timing, resource, fault, comm)
  test_type     ENUM(black_box, white_box, gray_box)
  level         ENUM(unit, integration, system)
  priority      ENUM(critical, high, medium, low)
  preconditions TEXT
  steps         JSONB  -- [{ order, action, expected }]
```

```
  expected_timing JSONB  -- { max_latency_us, deadline_ms }
  is_automated  BOOLEAN DEFAULT FALSE
  automation_script TEXT  -- path/code cho automated TC
```

2.2 Tập Dữ Liệu Kết Quả Kiểm Thử

Schema — Test Results

```
TABLE: test_runs
  run_id        UUID PK
  suite_id      UUID FK
  run_code      VARCHAR(30)  -- 'RUN-2024-001'
  hw_config     JSONB  -- { board, freq_mhz, flash_kb, ram_kb }
  fw_version    VARCHAR(50)
  rtos_version  VARCHAR(50)
  scheduler_algo ENUM(round_robin, priority, edf, rm)
  status        ENUM(running, passed, failed, aborted)
  total_cases   INT, passed INT, failed INT, skipped INT
  started_at    TIMESTAMPTZ, finished_at TIMESTAMPTZ
  tester_id     UUID FK → users

TABLE: test_results
  result_id     UUID PK
  run_id        UUID FK
  tc_id         UUID FK
  status        ENUM(pass, fail, skip, error)
  actual_output TEXT
  error_message TEXT
  execution_time_us INT  -- microseconds
  max_latency_us    INT
  deadline_missed   BOOLEAN DEFAULT FALSE
  context_switches  INT
  cpu_usage_pct     NUMERIC(5,2)
  memory_used_kb    INT
  stack_watermark   INT  -- stack high-water mark
  raw_log           TEXT  -- serial output
  metadata          JSONB
  executed_at       TIMESTAMPTZ
```

2.3 Time-Series Metrics (InfluxDB / TimescaleDB)

Time-Series Schema — Task Metrics

```
-- InfluxDB measurement: task_metrics
tags:
  run_id, task_name, priority_level

fields:
  execution_time_us   FLOAT  -- thời gian thực thi mỗi lần activate
  waiting_time_us     FLOAT  -- thời gian chờ trong ready queue
  cpu_load_pct        FLOAT  -- % CPU của task này
  stack_remaining_b   INT    -- stack còn lại
  preemption_count    INT    -- số lần bị preempt
  deadline_miss       BOOL

timestamp:  NANOSECOND precision

-- Query: deadline miss rate theo thời gian
SELECT count(deadline_miss) / count(*) as miss_rate
```

```
FROM task_metrics WHERE run_id=$run AND deadline_miss=true
GROUP BY time(1s), task_name
```

3. Tính Năng

3.1 Timing Analysis & Visualization

- Gantt chart real-time: hiển thị task execution timeline với preemption points
- WCET (Worst-Case Execution Time) analysis: tự động phát hiện outliers
- Response time distribution: histogram + percentile (p50, p95, p99)
- Deadline miss prediction: ML model cảnh báo sớm khi CPU load tăng

3.2 Anomaly Detection Engine

- Statistical process control: phát hiện drift trong execution time
- Stack overflow prediction: cảnh báo khi stack watermark < threshold
- Race condition detection: analyze log patterns của shared resource access
- Deadlock detection: timeout pattern analysis + resource dependency graph

3.3 Chatbot Test Assistant

- 'Test case nào đang fail nhiều nhất?' → aggregation query
- 'Phân tích nguyên nhân deadline miss của task Sensor?' → log analysis
- 'Gợi ý test case cho thuật toán Round Robin với 5 task' → TC generation

4. Dữ Liệu Thực

Loại data	Target	Nguồn
test_cases	500+	Xây dựng thủ công + import từ paper benchmark
test_results	50,000+	Chạy thực tế trên hardware + QEMU emulation
task_metrics (time-series)	10M+ rows	Sampling 1kHz × 10h chạy test
test_suites	20+	Theo từng scheduling algorithm